

DATA 260 Homework 5

Kavan Siddesh

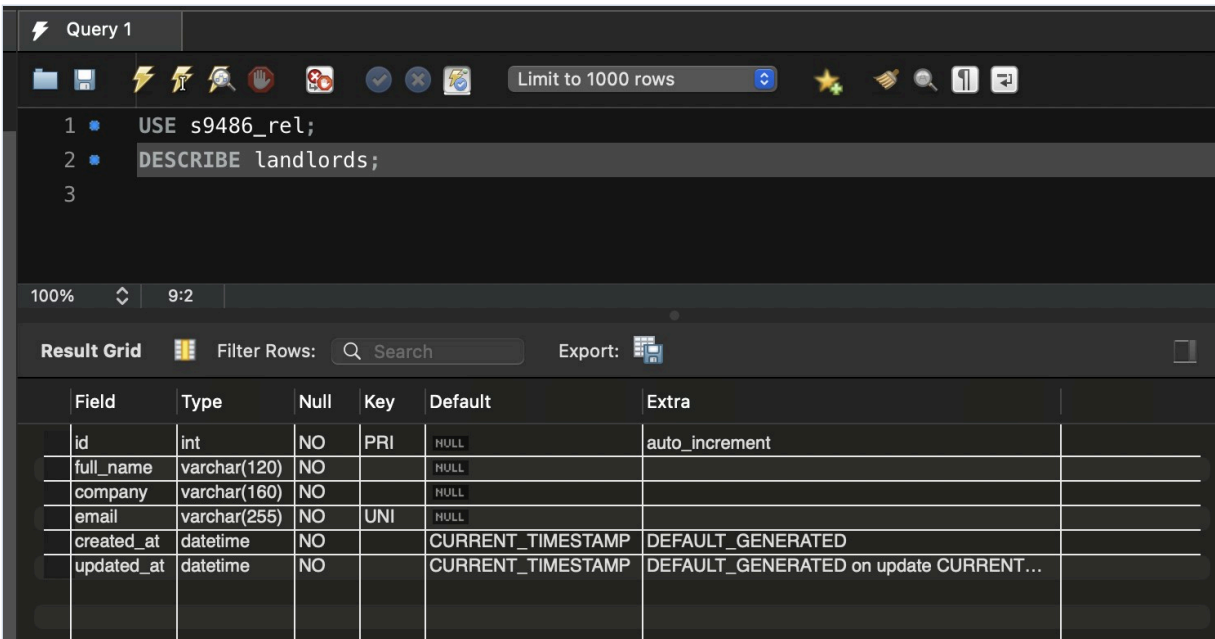
Section 0. Personal Configuration and Domain

GitHub repository	https://github.com/Kavan1804/data260-9486
SID4	9486
PORT_BASE	8486
PREFIX	s9486
SEED	9486
VERIFY_SEED	269486
DOMAIN_ID	6 - Rental housing listings
Hardware	Apple Mac with M1 chip and 8 GB unified memory
Local model	llama3.2:3b (Ollama)
Tagged commit hash	6c953745963b0351f744fee37eacab2b162bf46a (tag hw5)

Both collaborators (Sbnikitha, supriyaselvangesan) have access to the repository.

Part 1. FastAPI + MySQL Extension

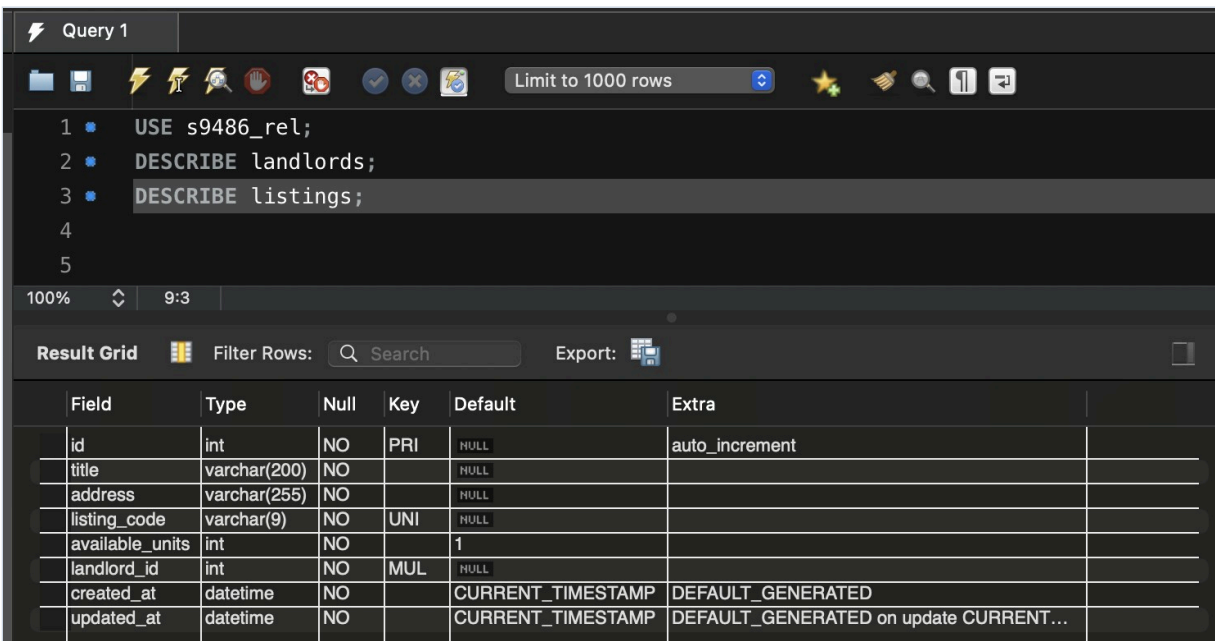
I. Database



The screenshot shows a database client interface with a query editor at the top containing two SQL statements: `USE s9486_rel;` and `DESCRIBE landlords;`. Below the editor, a toolbar includes icons for file operations, a 'Limit to 1000 rows' dropdown, and other utility icons. The main area displays the 'Result Grid' for the 'landlords' table. The grid has columns for Field, Type, Null, Key, Default, and Extra. The table structure is as follows:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
full_name	varchar(120)	NO		NULL	
company	varchar(160)	NO		NULL	
email	varchar(255)	NO	UNI	NULL	
created_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
updated_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT...

Figure 1. landlords table (related entity). (p1_db_landlords.png)



The screenshot shows a database client interface with a query editor at the top containing three SQL statements: `USE s9486_rel;`, `DESCRIBE landlords;`, and `DESCRIBE listings;`. Below the editor, a toolbar includes icons for file operations, a 'Limit to 1000 rows' dropdown, and other utility icons. The main area displays the 'Result Grid' for the 'listings' table. The grid has columns for Field, Type, Null, Key, Default, and Extra. The table structure is as follows:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
title	varchar(200)	NO		NULL	
address	varchar(255)	NO		NULL	
listing_code	varchar(9)	NO	UNI	NULL	
available_units	int	NO		1	
landlord_id	int	NO	MUL	NULL	
created_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
updated_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT...

Figure 2. listings table (primary entity) with landlord_id foreign key. (p1_db_listings.png)

II. API (Postman)

Landlords

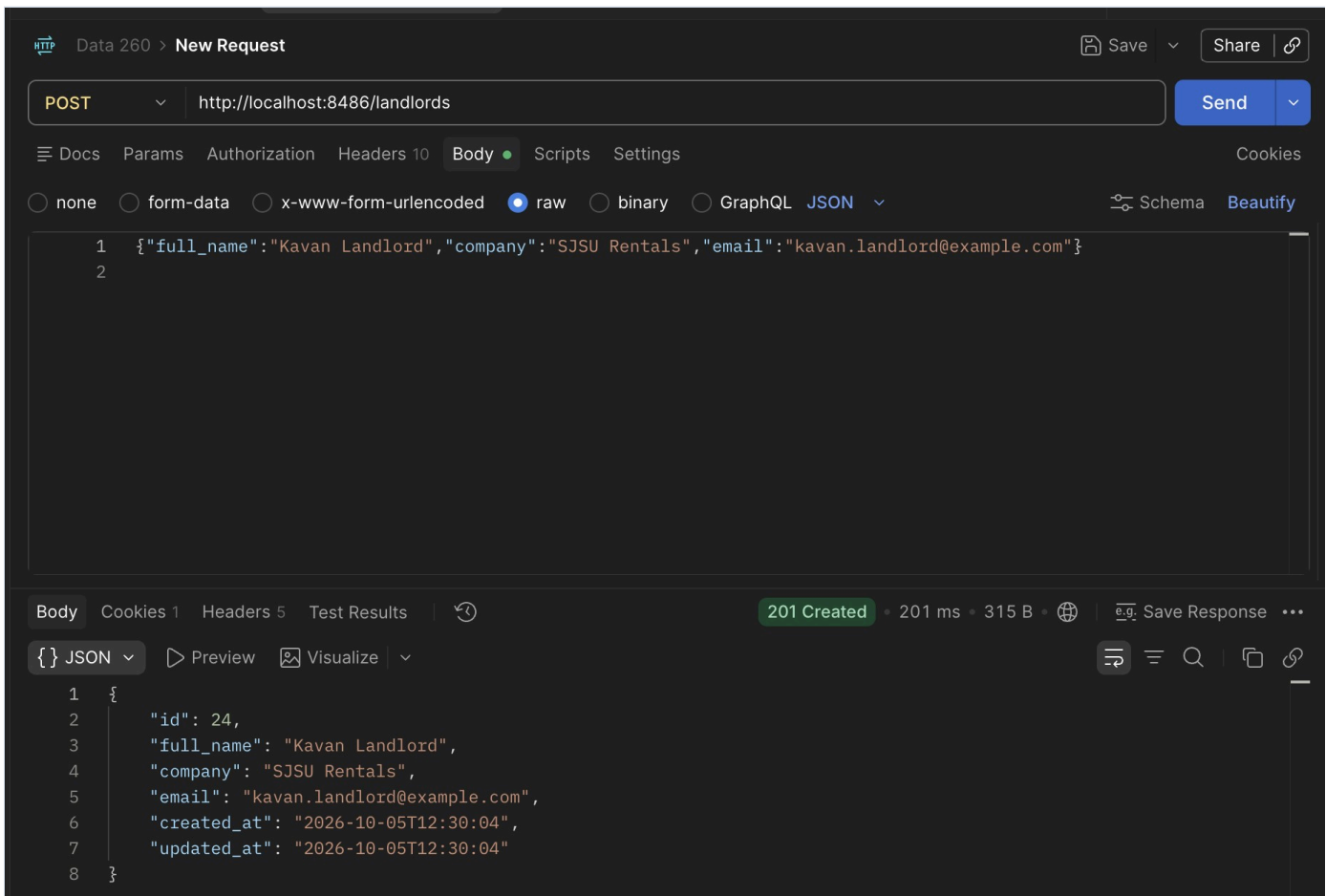


Figure 3. Create landlord - 201. (p1_landlord_create.png)

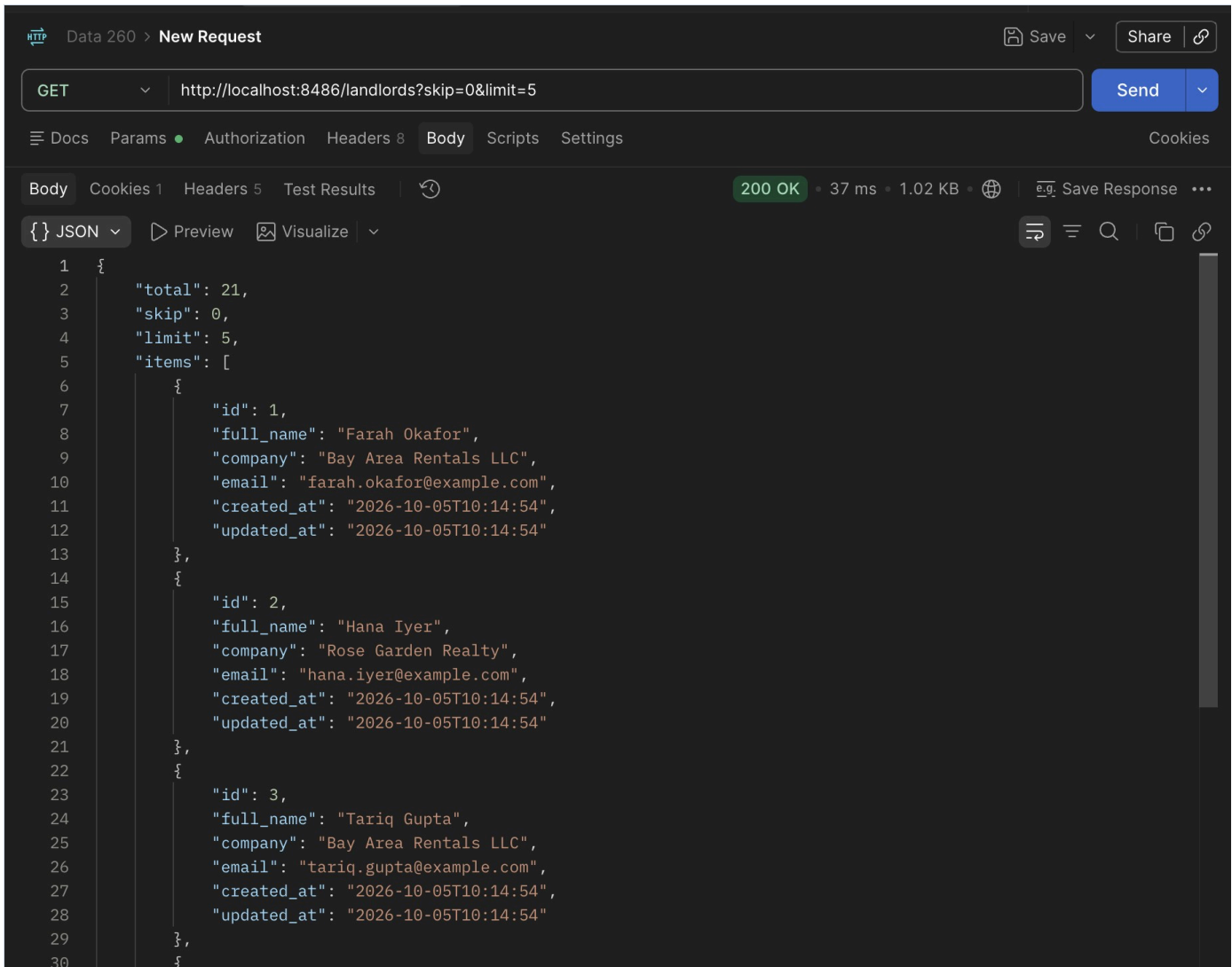


Figure 4. Get all landlords with pagination - 200. (p1_landlord_list.png)

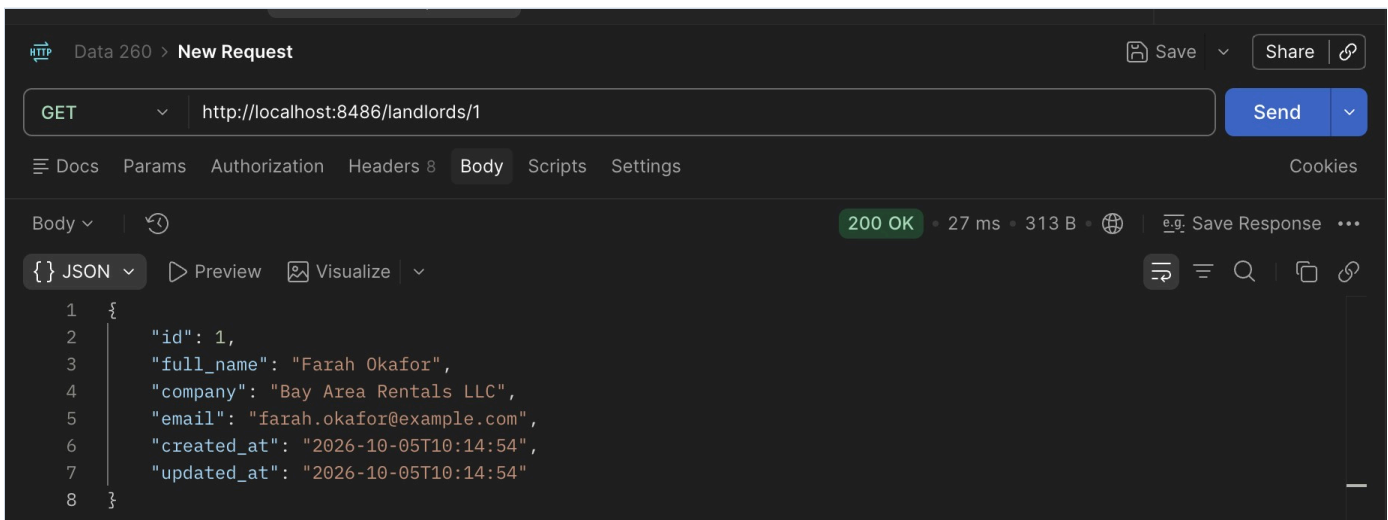


Figure 5. Get one landlord - 200. (p1_landlord_get.png)

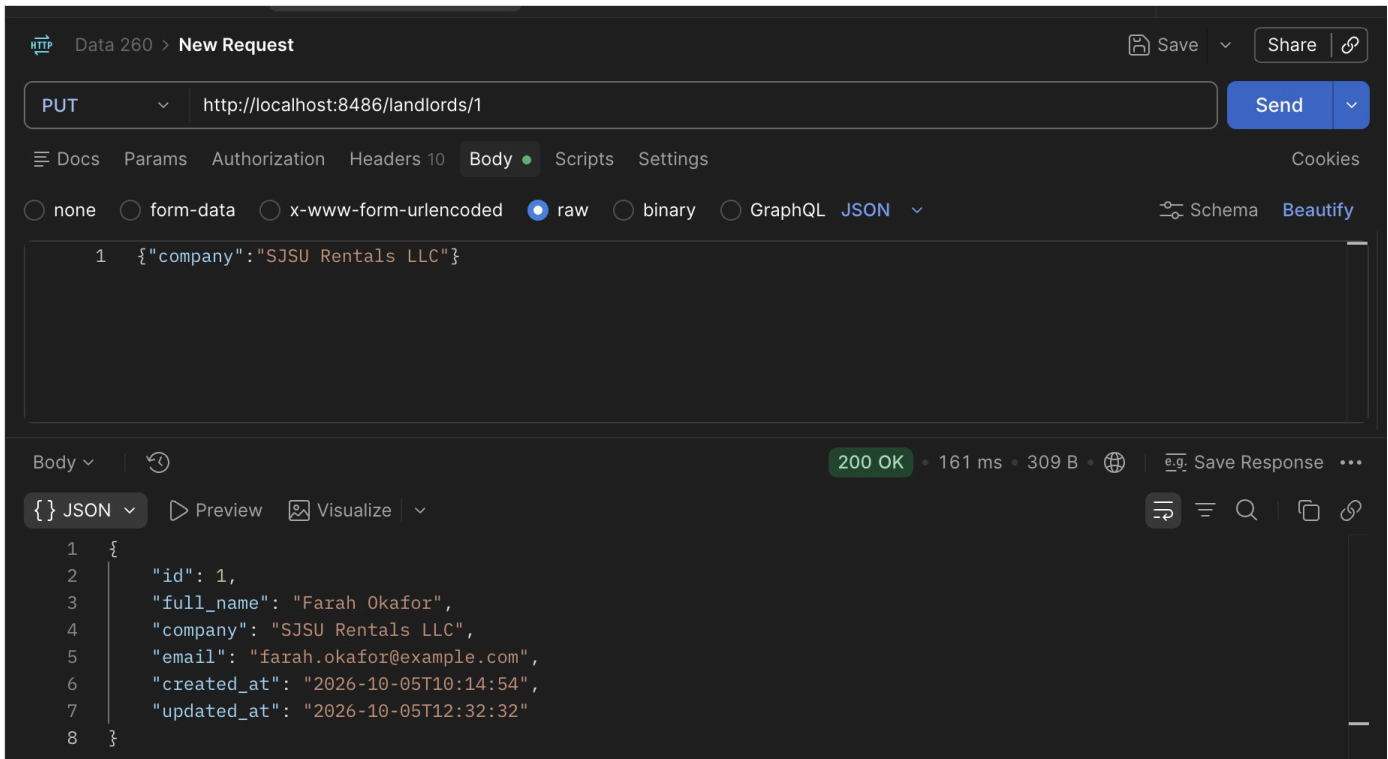


Figure 6. Update landlord - 200. (p1_landlord_update.png)

Listings

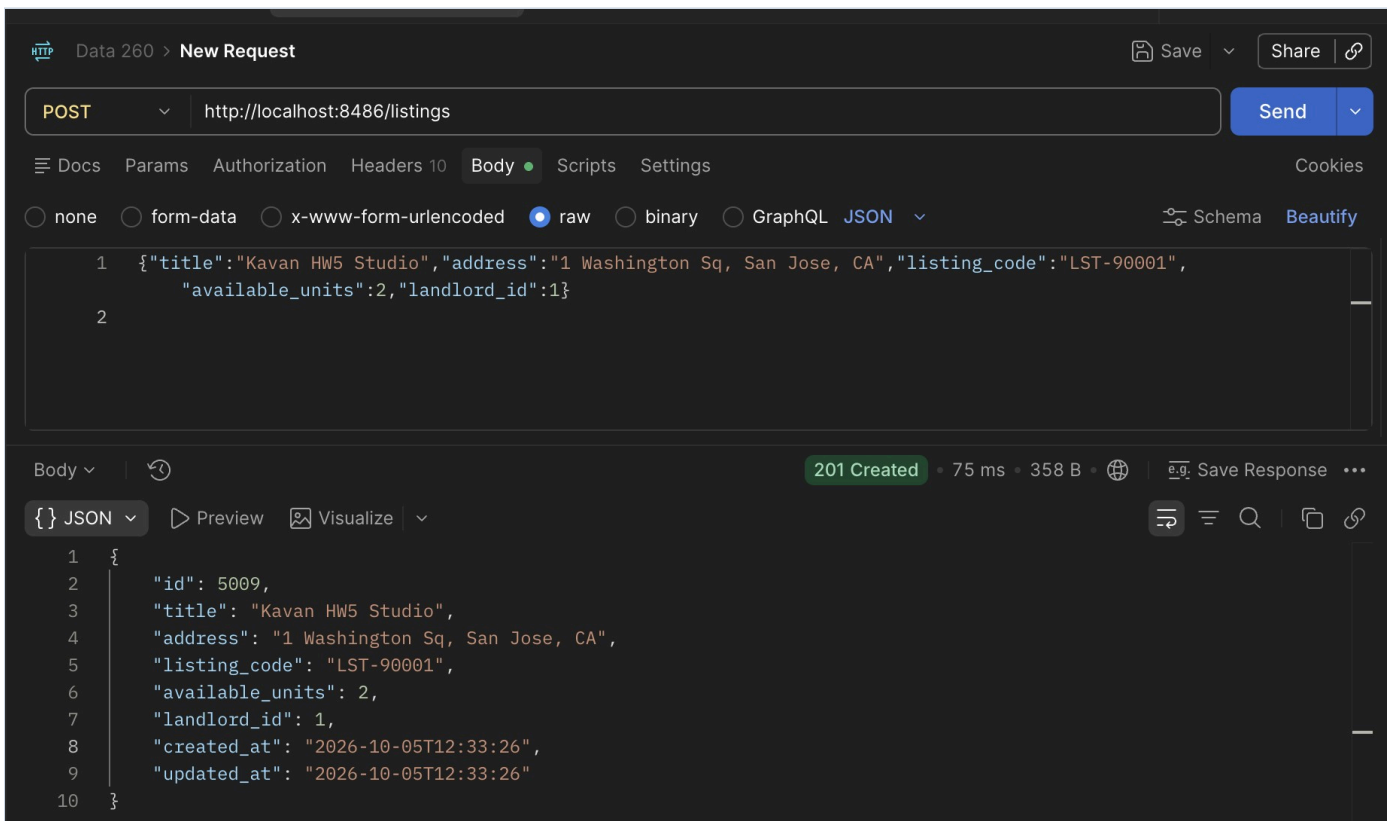


Figure 7. Create listing - 201. (p1_listing_create.png)

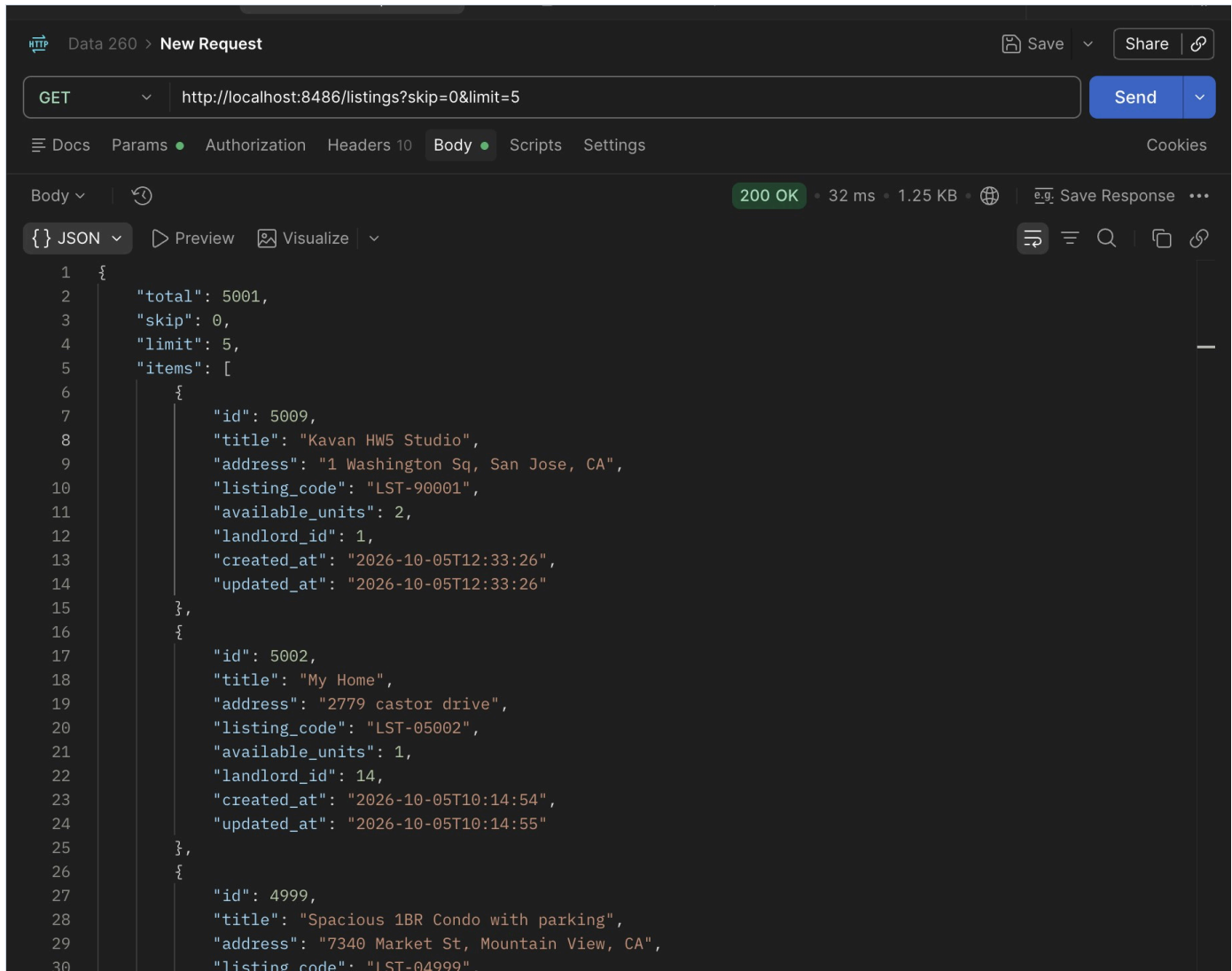


Figure 8. Get all listings with pagination - 200. (p1_listing_list.png)

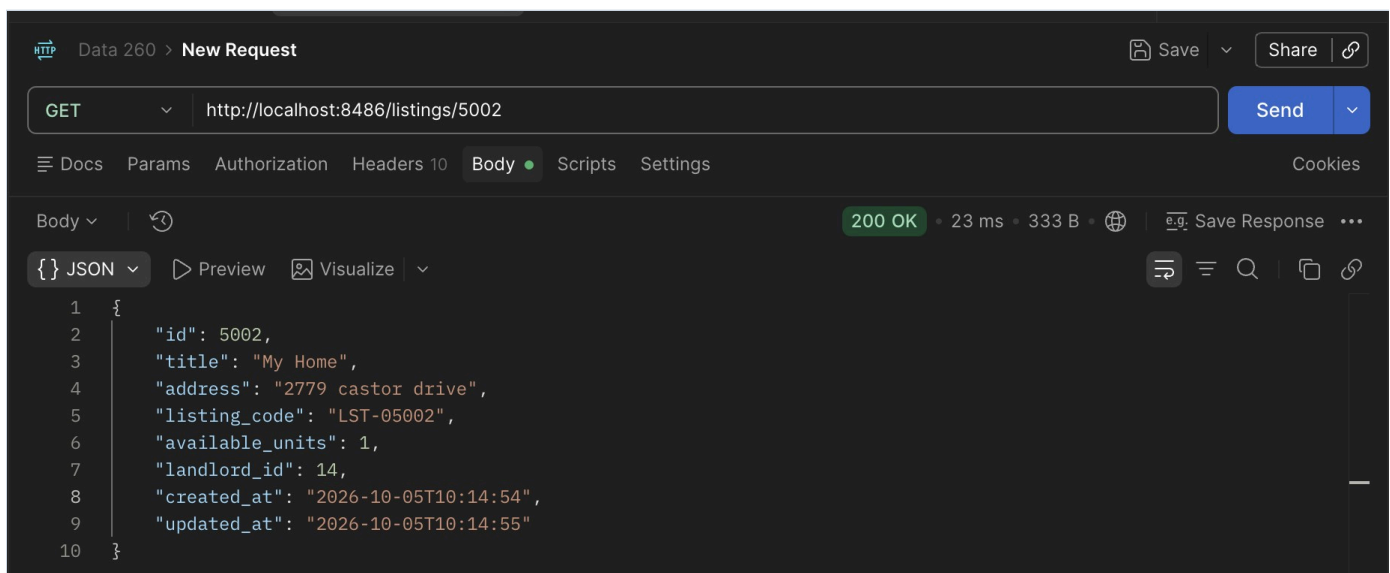


Figure 9. Get one listing - 200. (p1_listing_get.png)

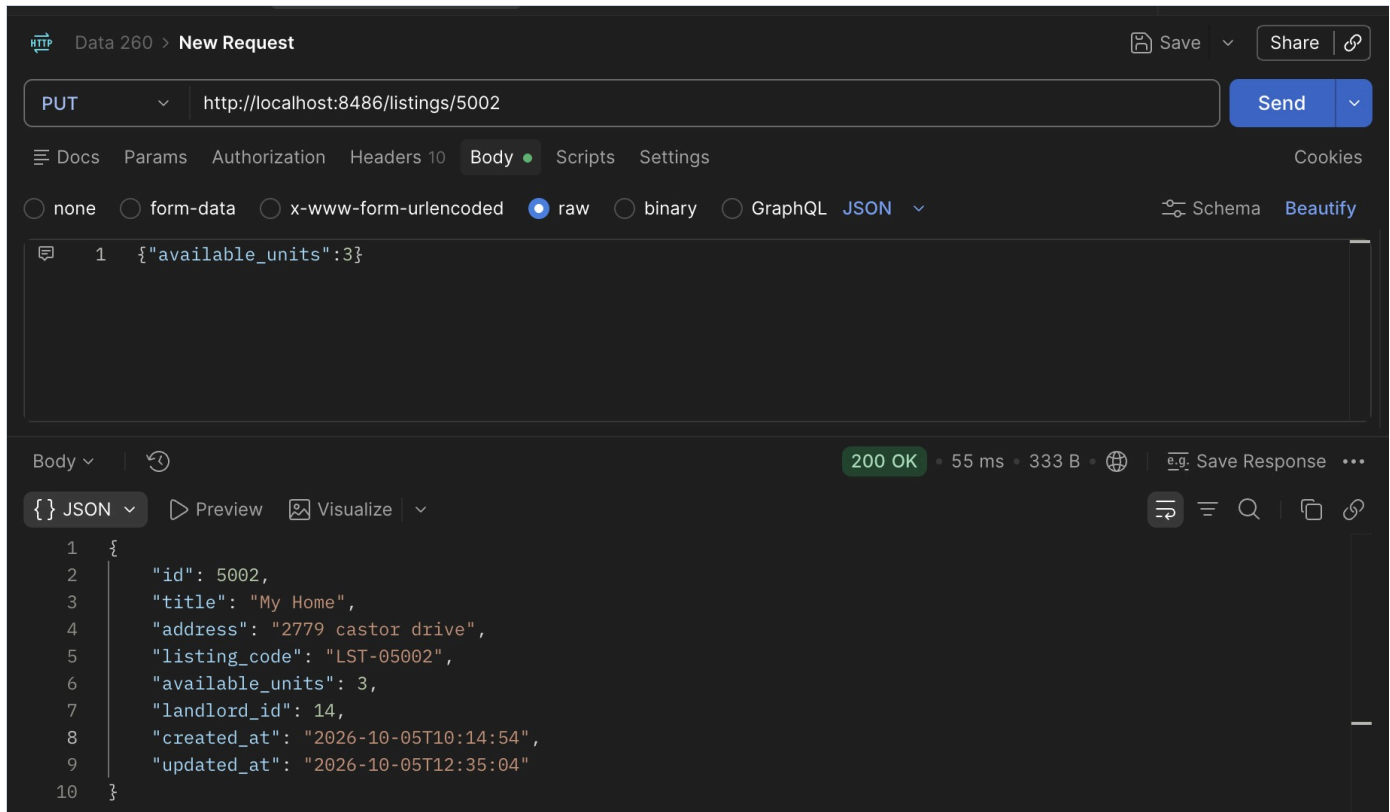


Figure 10. Update listing - 200. (p1_listing_update.png)

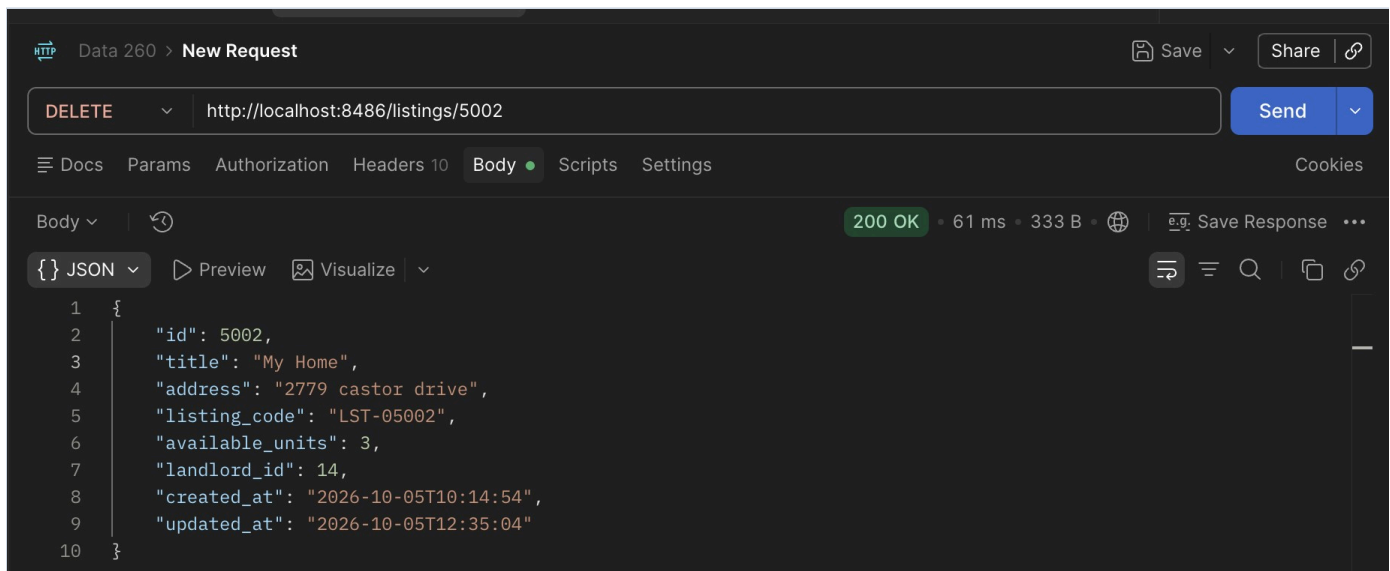
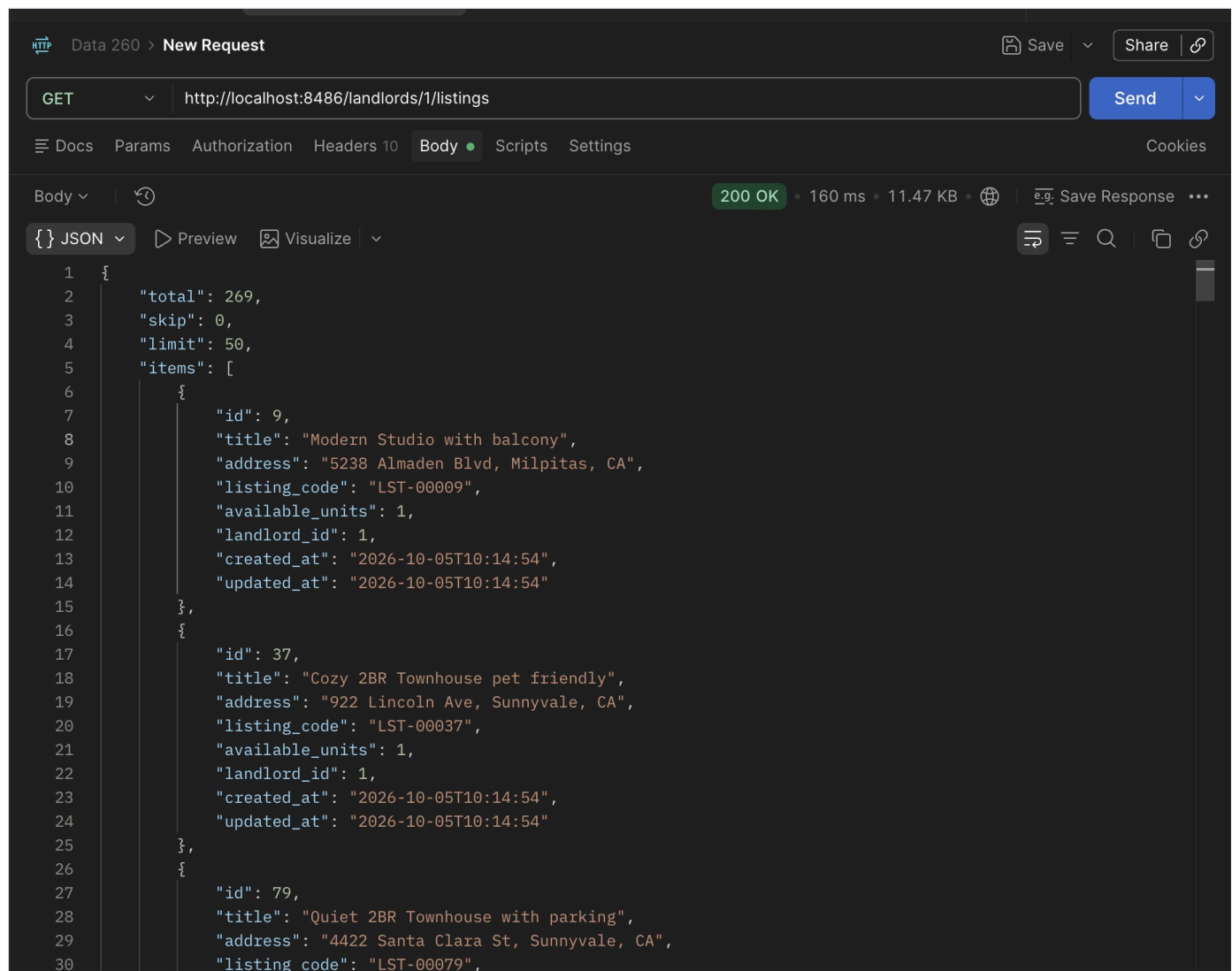


Figure 11. Delete listing - 200. (p1_listing_delete.png)

Relationship query and errors



The screenshot shows an HTTP client interface with a GET request to `http://localhost:8486/landlords/1/listings`. The response is a JSON array of 3 listings for landlord 1. The status is 200 OK, with a response time of 160 ms and a size of 11.47 KB.

```
{
  "total": 269,
  "skip": 0,
  "limit": 50,
  "items": [
    {
      "id": 9,
      "title": "Modern Studio with balcony",
      "address": "5238 Almaden Blvd, Milpitas, CA",
      "listing_code": "LST-00009",
      "available_units": 1,
      "landlord_id": 1,
      "created_at": "2026-10-05T10:14:54",
      "updated_at": "2026-10-05T10:14:54"
    },
    {
      "id": 37,
      "title": "Cozy 2BR Townhouse pet friendly",
      "address": "922 Lincoln Ave, Sunnyvale, CA",
      "listing_code": "LST-00037",
      "available_units": 1,
      "landlord_id": 1,
      "created_at": "2026-10-05T10:14:54",
      "updated_at": "2026-10-05T10:14:54"
    },
    {
      "id": 79,
      "title": "Quiet 2BR Townhouse with parking",
      "address": "4422 Santa Clara St, Sunnyvale, CA",
      "listing_code": "LST-00079",

```

Figure 12. All listings of landlord 1 - 200. (p1_relationship.png)

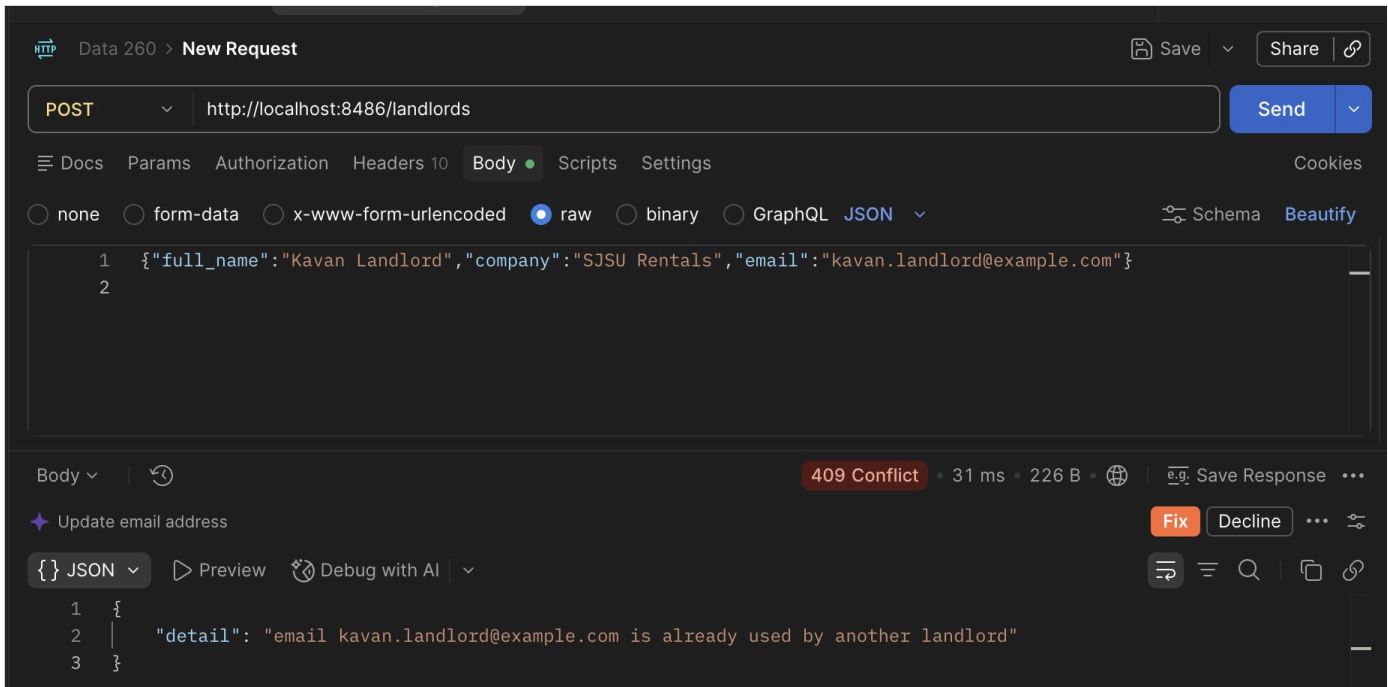


Figure 13. Duplicate email (unique constraint) - 409. (p1_err_duplicate_409.png)

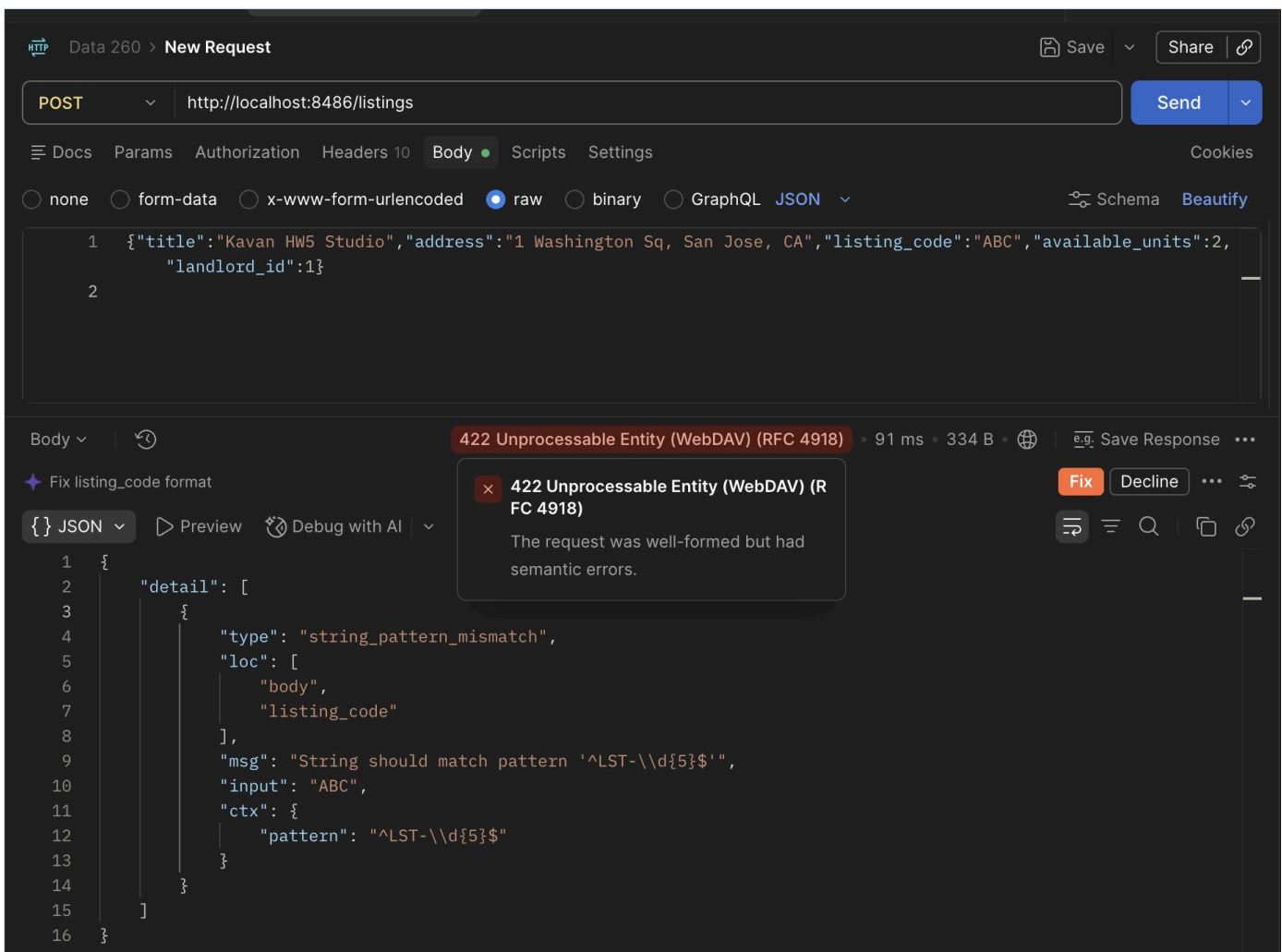


Figure 14. Invalid listing_code format (validation) - 422. (p1_err_format_422.png)

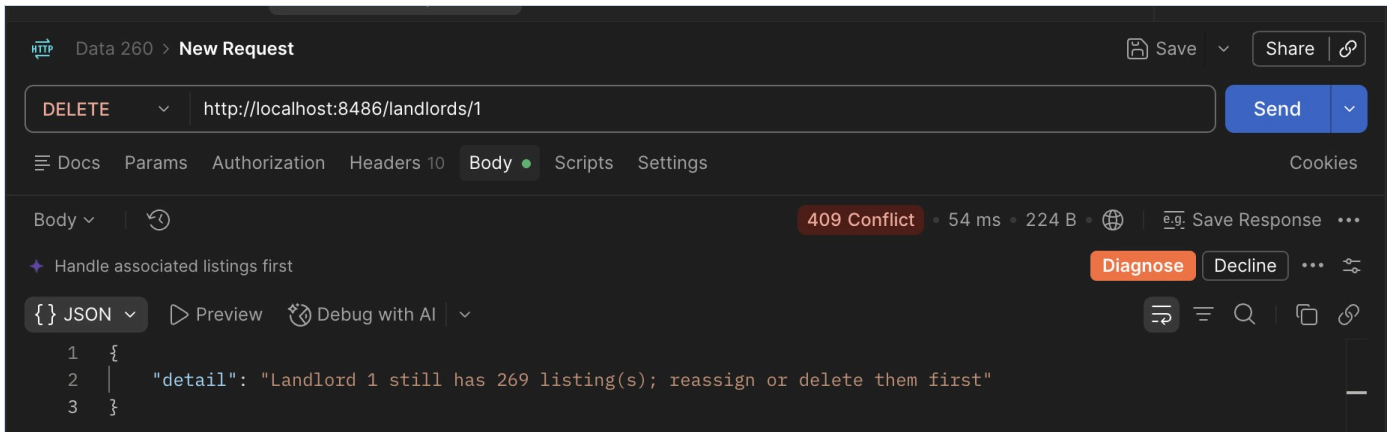


Figure 15. Delete landlord that still has listings - 409. (p1_err_restrict_409.png)

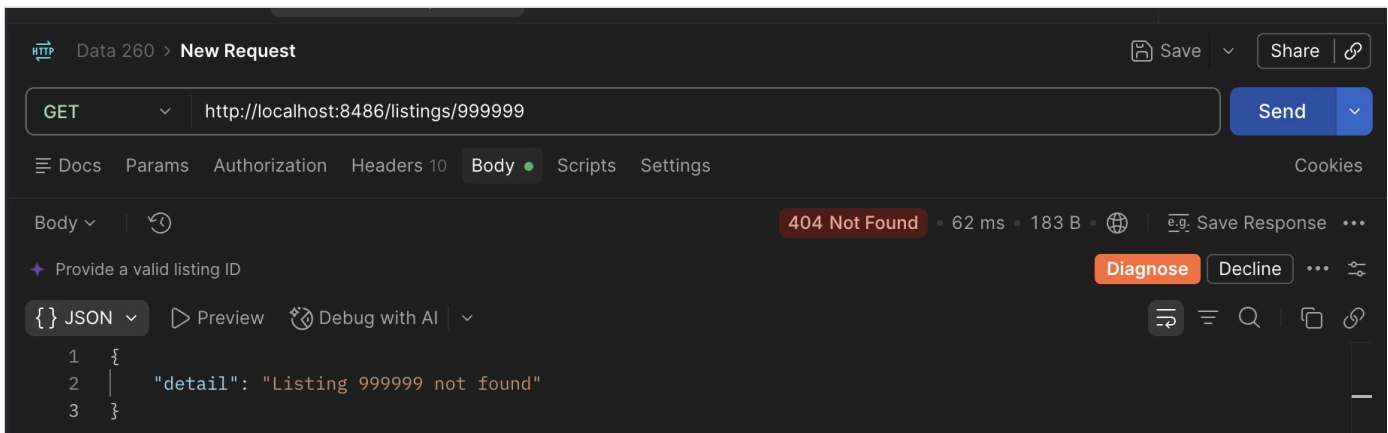


Figure 16. Listing not found - 404. (p1_err_notfound_404.png)

III. Redux Client

The image shows a code editor on the left and a web browser on the right. The code editor displays the `listingsSlice.js` file, which defines the Redux slice for rental listings. The browser shows the application running at `localhost:5173`, displaying the 'Rental Listings' page. The page includes navigation links (Home, Add Listing, Update Listing), a login status (Logged in as Kavan), and a table of listings.

```
import { createAsyncThunk, createSlice } from '@reduxjs/toolkit';
import { api, errorMessage } from '../api/listingsApi';

export const PAGE_SIZE = 25;

// Every thunk rejects with { message, status } so the UI can show the FastAPI error.
const reject = (thunkAPI, err) => {
  thunkAPI.rejectWithValue({ message: errorMessage(err), status: err?.response?.status ?? null });
};

export const fetchListings = createAsyncThunk('listings/fetch', async ({ skip = 0 } = {}, thunkAPI) => {
  try {
    const res = await api.get('/listings', { params: { skip, limit: PAGE_SIZE } });
    return res.data; // { total, skip, limit, items }
  } catch (err) {
    return reject(thunkAPI, err);
  }
});

export const createListing = createAsyncThunk('listings/create', async (payload, thunkAPI) => {
  try {
    const res = await api.post('/listings', payload);
    return res.data;
  } catch (err) {
    return reject(thunkAPI, err);
  }
});

export const updateListing = createAsyncThunk('listings/update', async ({ id, changes }, thunkAPI) => {
  try {
    const res = await api.put(`/listings/${id}`, changes);
    return res.data;
  } catch (err) {
    return reject(thunkAPI, err);
  }
});

export const deleteListing = createAsyncThunk('listings/delete', async (id, thunkAPI) => {
  try {
    await api.delete(`/listings/${id}`);
    return id;
  } catch (err) {
    return reject(thunkAPI, err);
  }
});

const initialState = {
  items: [],
  total: 0,
  skip: 0,
  status: 'idle', // idle | loading | succeeded | failed
  error: null,
  notice: null, // last success message shown on Home
};

const listingsSlice = createSlice({
  name: 'listings',
  initialState,
  reducers: {
    clearMessages(state) {
      state.error = null;
      state.notice = null;
    },
    resetListings() {
      return initialState;
    },
  },
  extraReducers: (builder) => {
    builder
      .addCase(fetchListings.pending, (s) => {
        s.status = 'loading';
        s.error = null;
      })
      .addCase(fetchListings.fulfilled, (s, a) => {
        s.status = 'succeeded';
        s.items = a.payload.items;
        s.total = a.payload.total;
        s.skip = a.payload.skip;
      })
      .addCase(fetchListings.rejected, (s, a) => {
        s.status = 'failed';
        s.error = a.payload.message;
      });
  },
});

export default listingsSlice.reducer;
```

The browser shows the 'Rental Listings' page with the following content:

- Navigation: Home, Add Listing, Update Listing
- Login Status: Logged in as Kavan (kavansiddesh02@gmail.com) with a Logout button
- Section: Rental Listings
- Text: Redux store: showing 1-25 of 5000 listings from FastAPI + MySQL (s9486_rel).
- + Add Listing button
- Table of listings:

ID	CODE	TITLE	ADDRESS	UNITS
5009	LST-90001	Kavan HW5 Studio	1 Washington Sq, San Jose, CA	2
4999	LST-04999	Spacious 1BR Condo with parking	7340 Market St, Mountain View, CA	1

Figure 17. Home - fetchListings thunk/slice and the list read from Redux state. (p1_redux_home.png)

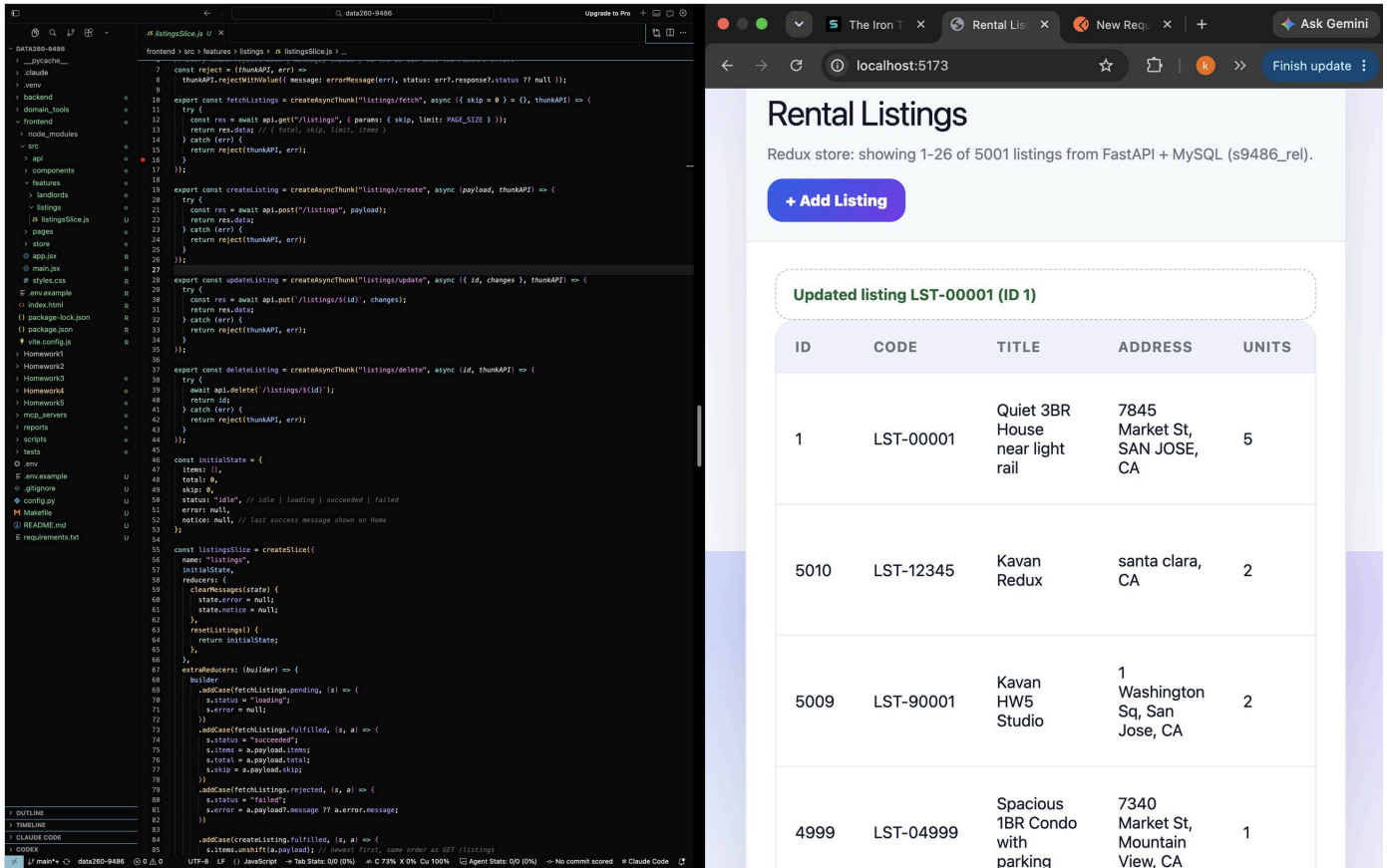


Figure 19. Update - updateListing thunk and the updated listing on Home. (p1_redux_update.png)

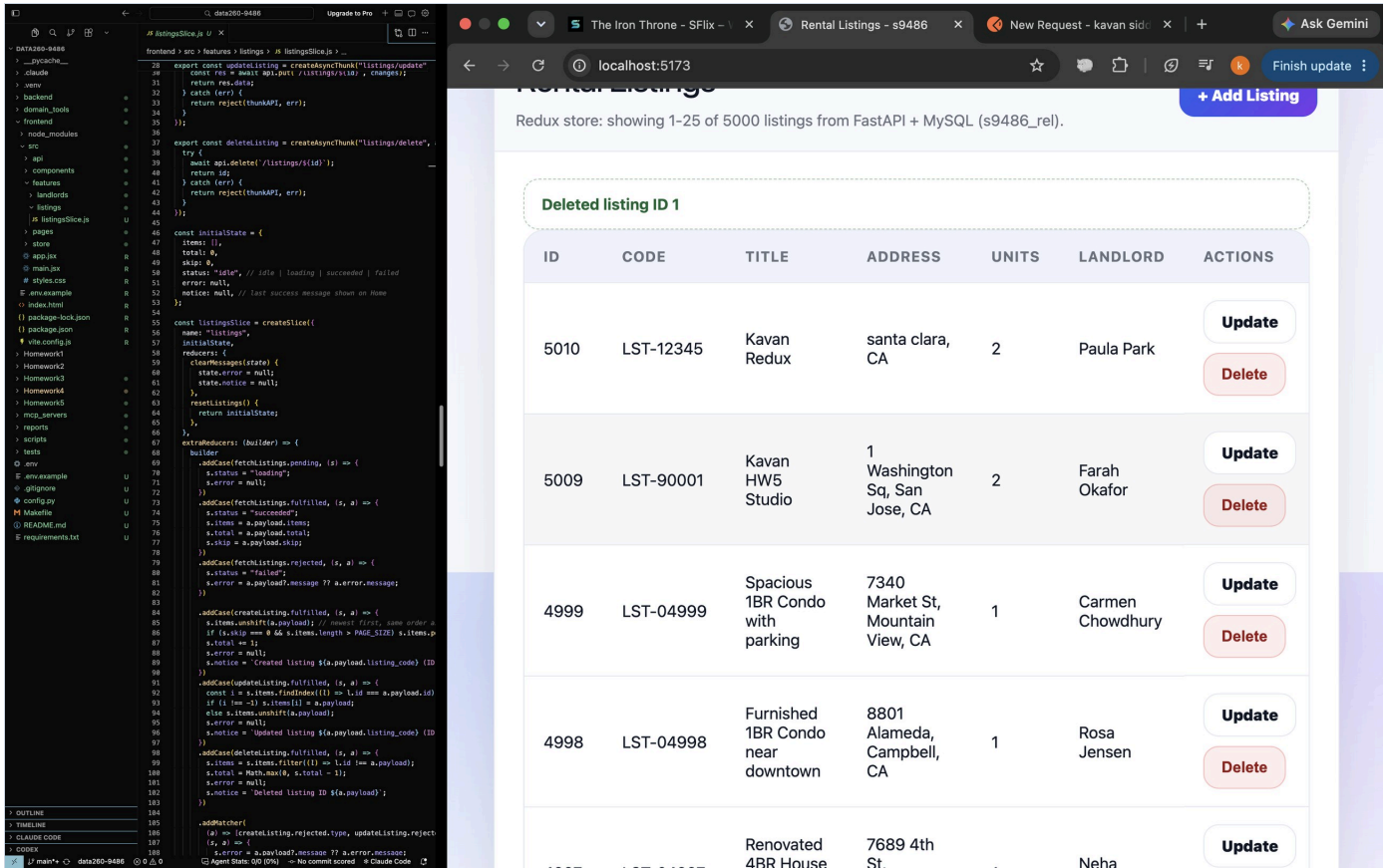


Figure 20. Delete - deleteListing thunk and the listing removed from Home. (p1_redux_delete.png)

Part 2. MCP Tool Servers

A. TheMealDB Server (meals)

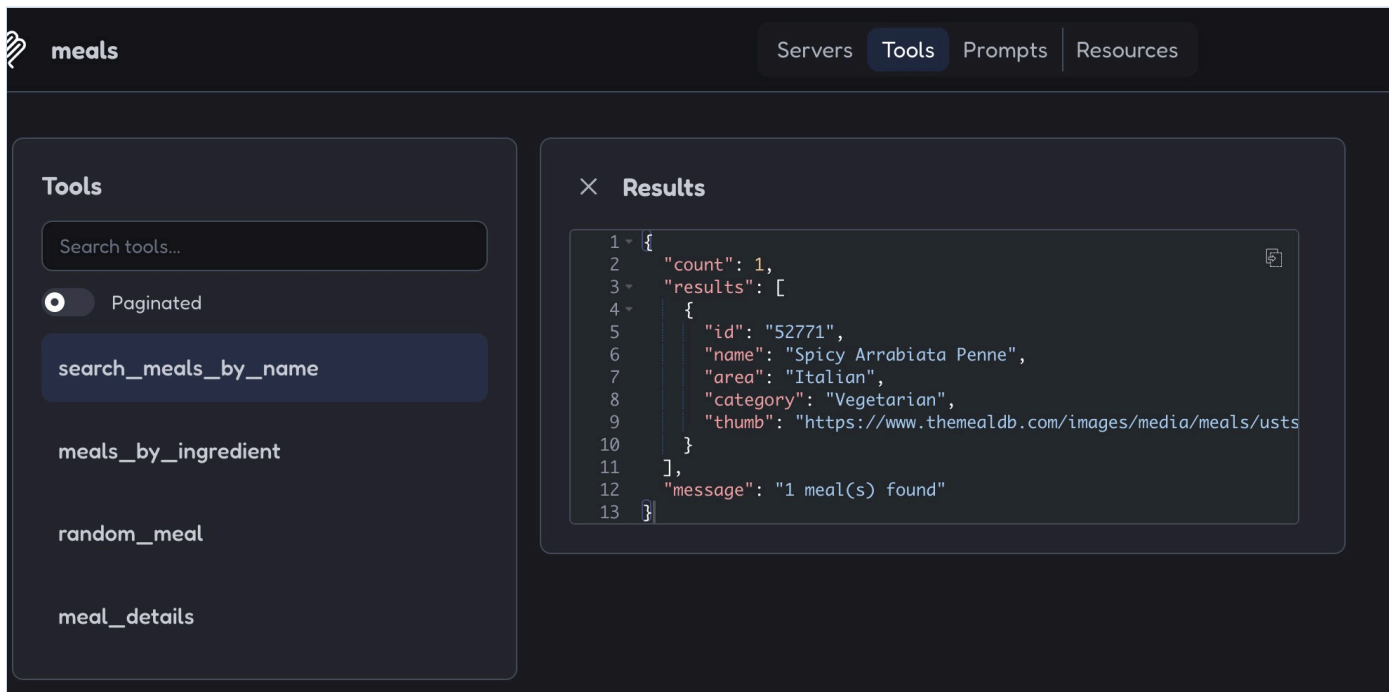


Figure 21. search_meals_by_name - query "Arrabiata", limit 5. (p2_meals_search.png)

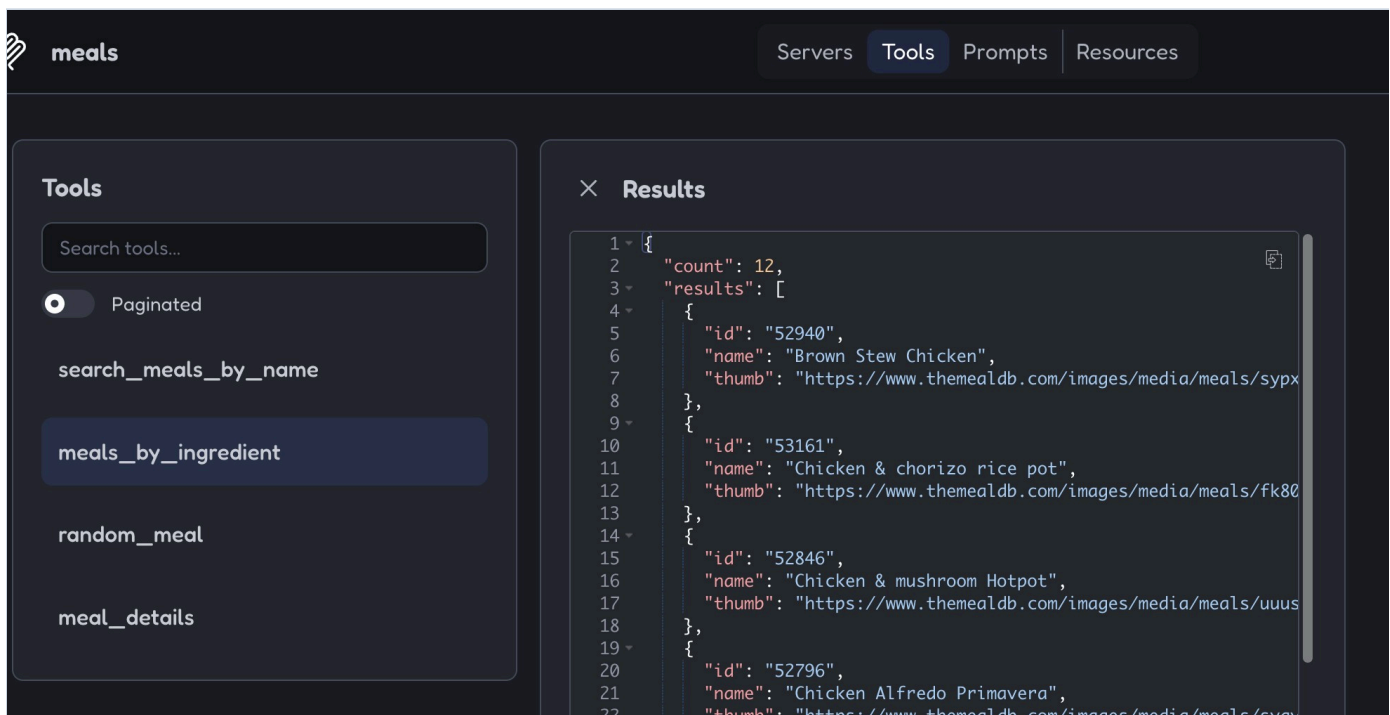


Figure 22. meals_by_ingredient - ingredient "chicken", limit 12. (p2_meals_ingredient.png)

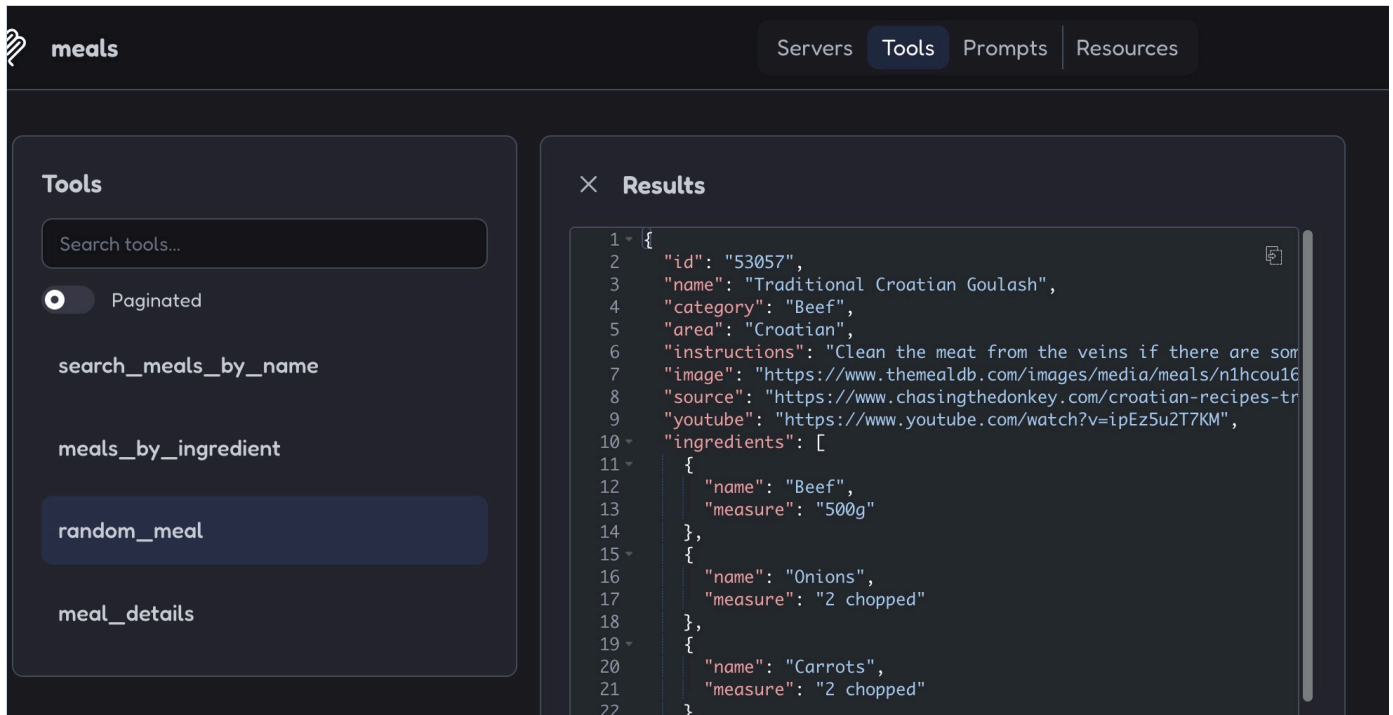


Figure 23. random_meal. (p2_meals_random.png)

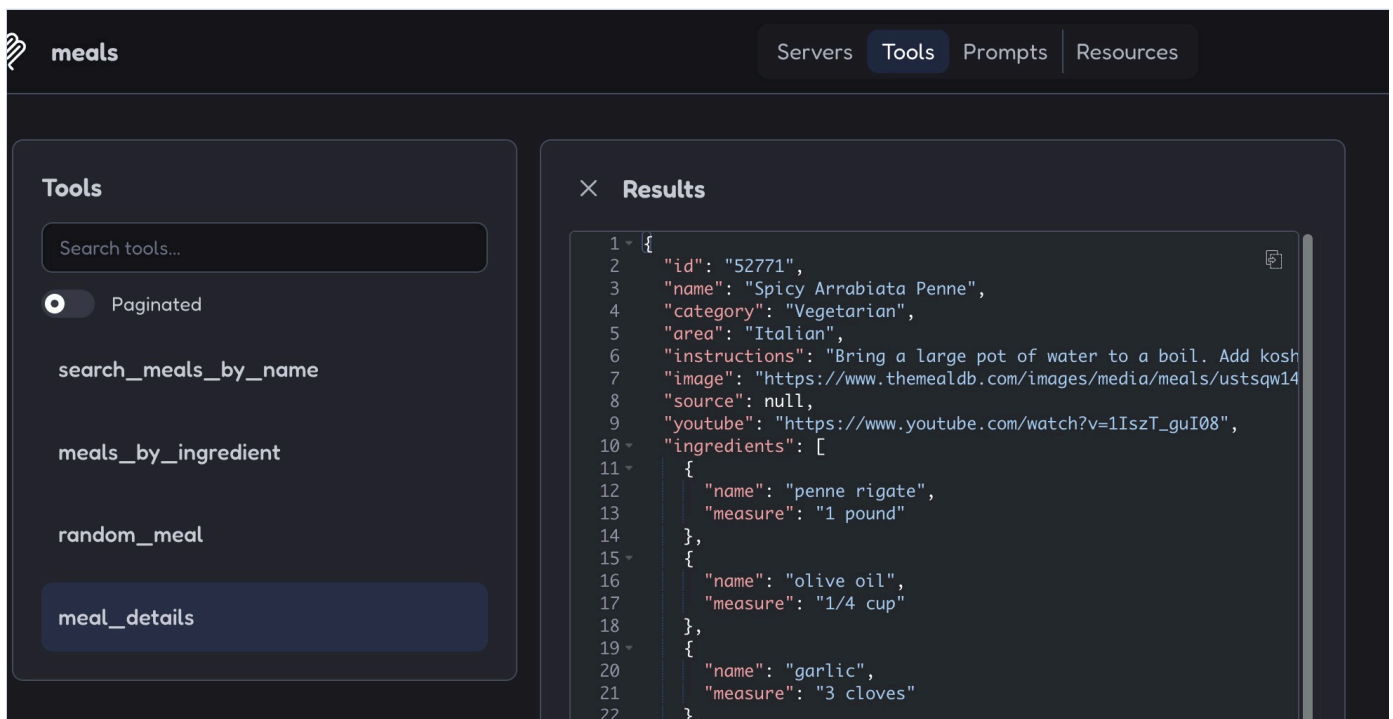


Figure 24. meal_details - id 52771. (p2_meals_details.png)

B. Domain Server (s9486_listings)

The screenshot shows the s9486_listings domain server interface. On the left, the 'Tools' panel lists three available tools: 'search_listings', 'get_listing', and 'landlord_portfolio_stats'. The 'search_listings' tool is selected. On the right, the 'Results' panel displays the JSON response for a valid search_listings call. The response is a JSON object with 'ok' set to true, 'data' containing an array of three listing objects, and 'error' set to null. The listings are for 'Campbell', 'Sunny 3BR House with in-unit laundry', and 'Spacious 1BR Condo near light rail'.

```
1 {
2   "ok": true,
3   "data": {
4     "query": "Campbell",
5     "min_units": 0,
6     "count": 3,
7     "listings": [
8       {
9         "listing_code": "LST-00017",
10        "title": "Quiet Studio with yard",
11        "address": "2900 Santa Clara St, Campbell, CA",
12        "available_units": 1,
13        "landlord_id": 12,
14        "landlord_name": "Oscar Tran"
15      },
16      {
17        "listing_code": "LST-00023",
18        "title": "Sunny 3BR House with in-unit laundry",
19        "address": "1110 Willow St, Campbell, CA",
20        "available_units": 1,
21        "landlord_id": 19,
22        "landlord_name": "Kevin Dang"
23      },
24      {
25        "listing_code": "LST-00027",
26        "title": "Spacious 1BR Condo near light rail",
27        "address": "6788 Alameda, Campbell, CA",
28        "available_units": 1,
29        "landlord_id": 3,
30        "landlord_name": "Tariq Gupta"
31      }
32    ]
33  },
34  "error": null
35 }
```

Figure 25. search_listings - valid call. (p2_domain_search_ok.png)

The screenshot shows the s9486_listings domain server interface. On the left, the 'Tools' panel lists three available tools: 'search_listings', 'get_listing', and 'landlord_portfolio_stats'. The 'search_listings' tool is selected. On the right, the 'Results' panel displays the JSON response for an invalid search_listings call. The response is a JSON object with 'ok' set to false, 'data' set to null, and 'error' set to 'Invalid input for search_listings: query must be 2-80'.

```
1 {
2   "ok": false,
3   "data": null,
4   "error": "Invalid input for search_listings: query must be 2-80"
5 }
```

Figure 26. search_listings - invalid call. (p2_domain_search_invalid.png)

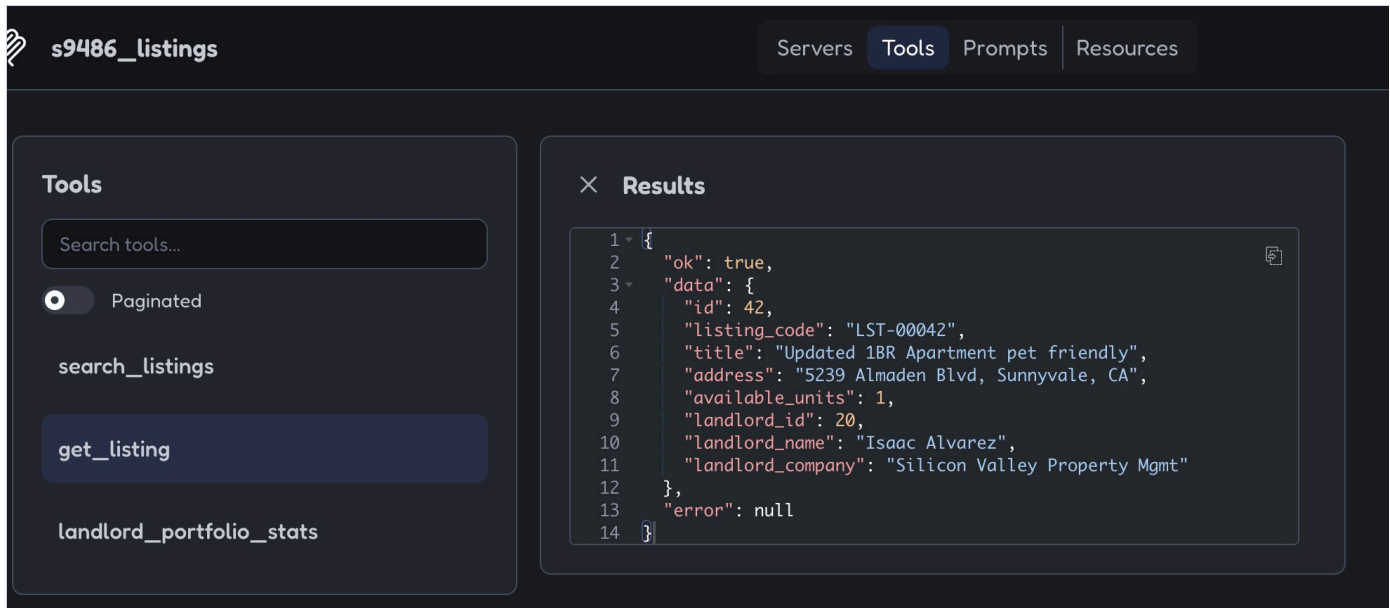


Figure 27. get_listing - valid call. (p2_domain_detail_ok.png)

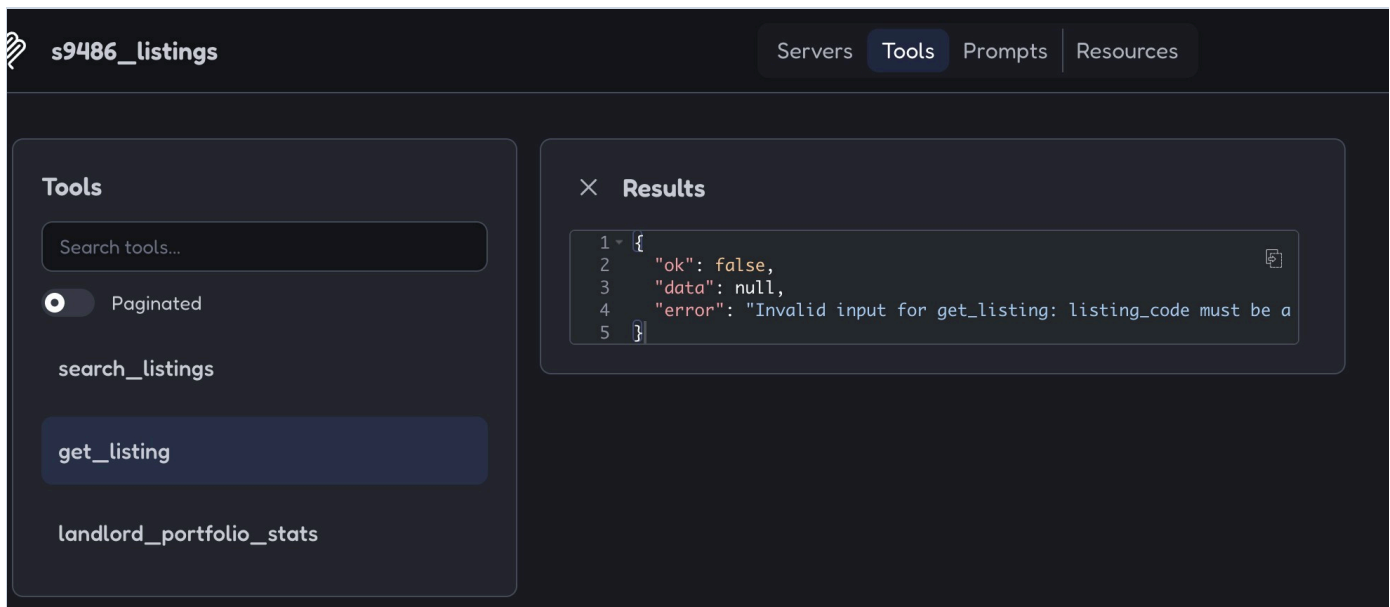


Figure 28. get_listing - invalid call. (p2_domain_detail_invalid.png)

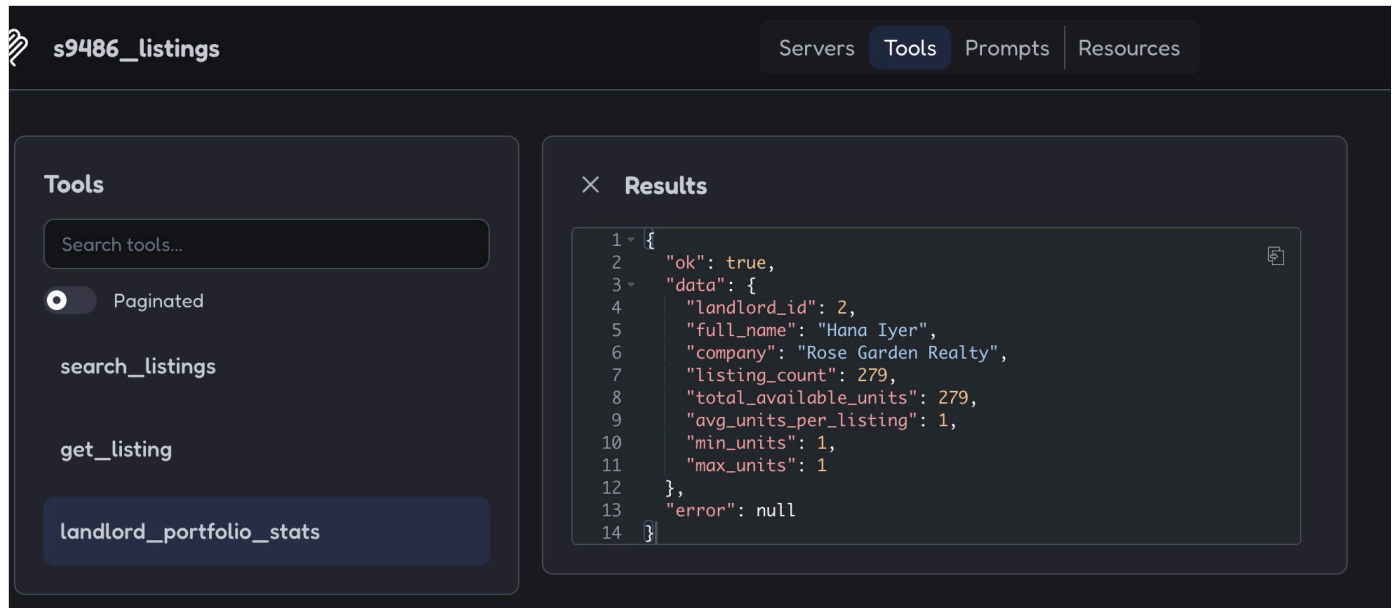


Figure 29. landlord_portfolio_stats - valid call. (p2_domain_stats_ok.png)

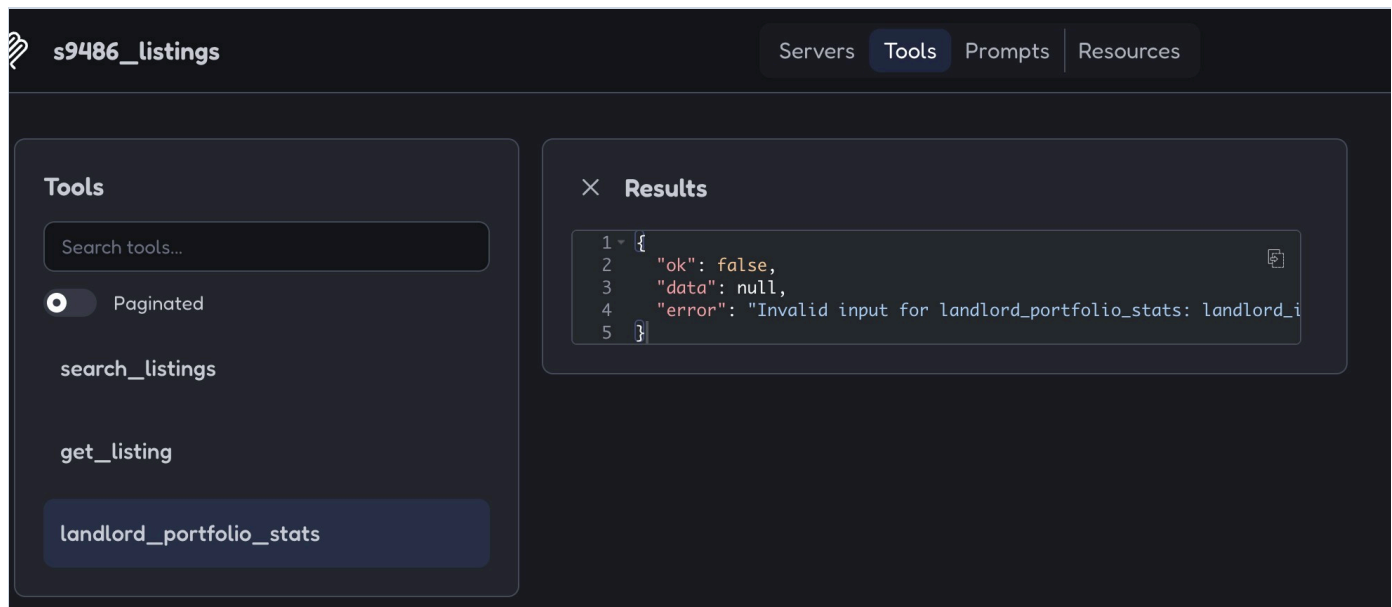


Figure 30. landlord_portfolio_stats - invalid call. (p2_domain_stats_invalid.png)

3. Failure simulation with VERIFY_SEED (50 calls per rate)

Injected failure rate	Success rate	Mean latency (ms)	p99 latency (ms)
0%	100%	0.85	2.24
20%	100%	15.34	77.19
50%	88%	48.22	165.03

```
● Kavan Siddesh data260-9486 % make retry-experiment

===== 2026-10-05 13:22:14 retry-experiment (Kavan Siddesh) =====
=====
Kavan Siddesh | SID4 9486 | s9486 | Part 3 - fault injection: 3 rates x 50 calls
run at 2026-10-05T13:22:14 | VERIFY_SEED 269486
=====
policy: max_attempts=3, backoff 50 ms x2 (cap 400 ms), timeout 2.0 s/attempt; seed=269486
rate 0%: success 50/50 (100%), attempts 50, injected failures 0, calls retried 0, mean 0.85 ms, p99 2.24 ms, decisions sha1 6c5e42666ed5
rate 20%: success 50/50 (100%), attempts 62, injected failures 12, calls retried 12, mean 15.34 ms, p99 77.19 ms, decisions sha1 b66750a3fdde
rate 50%: success 44/50 (88%), attempts 84, injected failures 40, calls retried 24, mean 48.22 ms, p99 165.03 ms, decisions sha1 2541868cf8ca

saved reports/hw05/raw/retry_calls.csv, retry_calls.json, retry_summary.json; updated METRICS.md
○ Kavan Siddesh data260-9486 %
```

Figure 32. 150 calls at 0%, 20% and 50% (raw data in reports/hw05/raw/). (p3_retry_experiment.png)

4. Evaluation

Interactive assistant: the policy is suitable. At 20% failures every call still succeeded with only 15 ms mean and 77 ms p99 latency. At 50% the success rate was 88% and p99 stayed at 165 ms, so failures come back quickly as clean errors. The backoff waits at most 150 ms in total. The only risk is the 2 s timeout per attempt (up to about 6 s if the database hangs), so for chat I would add an overall limit of about 2-3 s.

Batch processing: speed matters less than finishing every record, so I would use more retries (5-8), longer backoff with jitter (starting around 0.5-1 s, capped at 30-60 s) and longer timeouts (10-30 s), and log records that still fail so they can be retried later.

Part 4. One Safe Tool Entry Point and Offline Tests

```
● Kavan Siddesh data260-9486 % make test

===== 2026-10-05 13:22:35 test (Kavan Siddesh) =====
=====
Kavan Siddesh | SID4 9486 | s9486 | Part 4 + 5 offline tests (no DB, no LLM, no network)
run at 2026-10-05T13:22:35 | VERIFY_SEED 269486
=====
PASS 1. test_search_listings_valid
PASS 2. test_search_listings_rejected_input
PASS 3. test_get_listing_valid
PASS 4. test_get_listing_rejected_input
PASS 5. test_landlord_portfolio_stats_valid
PASS 6. test_landlord_portfolio_stats_rejected_input
PASS 7. test_not_found_is_clean_error
PASS 8. test_execute_tool_bad_name_and_inputs_do_not_crash
PASS 9. test_retry_first_attempt_success
PASS 10. test_retry_fail_then_success
PASS 11. test_retry_exhausted_returns_clean_error
PASS 12. test_backoff_delays_are_bounded_exponential
PASS 13. test_fault_injection_is_reproducible
PASS 14. test_safety_rule_blocks_violating_call
PASS 15. test_run_agent_mock_model_stops_at_max_steps
PASS 16. test_agent_rejects_fabricated_tool_result

16/16 tests passed
```

Figure 33. Offline test runner: PASS/FAIL for each test and the final summary. (p4_offline_tests.png)

Part 5. Agent Loop, Safety Rule, and Reflection

I. Safety rule

```
20/20 tests passed
Kavan Siddesh data260-9486 % make safety-demo

===== 2026-10-05 13:23:02 safety-demo (Kavan Siddesh) =====
Kavan Siddesh | SID4 9486 | s9486 | Part 5 - fair-housing safety rule in execute_tool
run at 2026-10-05T13:23:02 | VERIFY_SEED 269486
=====
ALLOWED: execute_tool('search_listings', {'query': "San Jose", "limit": 3})
-> ok=true, error=null, 3 listings: ['LST-00006', 'LST-00010', 'LST-00013']

BLOCKED: execute_tool('search_listings', {'query': "San Jose no kids", "limit": 3})
-> {'ok': false, "data": null, "error": "Blocked by fair-housing safety rule: listing searches cannot filter on a protected characteristic (familial status: 'no kids'). Search by location, title words, or available units instead."}

saved reports/hw05/raw/safety_demo.json
Kavan Siddesh data260-9486 %
```

Figure 34. Fair-housing rule in execute_tool: one allowed call and one blocked call. (p5_safety.png)

II. Agent loop

```
Kavan Siddesh data260-9486 % make agent-scenarios

===== 2026-10-05 13:23:19 agent-scenarios (Kavan Siddesh) =====
Kavan Siddesh | SID4 9486 | s9486 | Part 5 - agent scenarios with llama3.2:3b (ollama)
run at 2026-10-05T13:23:19 | VERIFY_SEED 269486
=====

[search] Find up to 3 rental listings in Campbell. (max_steps=6)
run_id 38b618638804: steps=2 tool_calls=1 stop_reason=completed (23.02 s)
answer: Here are 3 rental listings in Campbell:

1. Quiet Studio with yard - 2900 Santa Clara St, Campbell, CA (LST-00017)
2. Sunny 3BR House with in-unit laundry - 1110 Willow St, Campbell, CA (LST-00023)
3. Spacious 1BR Condo near light rail - 6788 Alameda, Campbell, CA (LST-00027)

[detail lookup] Give me the details for listing LST-00042. (max_steps=6)
run_id 082c7036c0be: steps=2 tool_calls=1 stop_reason=completed (7.62 s)
answer: Listing LST-00042 is a 1-bedroom apartment available for rent. It is located at 5239 Almaden Blvd, Sunnyvale, CA, and is pet-friendly. The landlord's name is Isaac Alvarez and the property management company is Silicon Valley Property Mgmt.

[aggregate] How many listings does landlord 3 manage, and how many units are available in total? (max_steps=6)
run_id 9bf0a8e13fc0: steps=2 tool_calls=1 stop_reason=completed (5.35 s)
answer: Landlord 3, Tariq Gupta, manages 233 listings. There are a total of 233 units available in his portfolio.

[safety rule] Find apartments in San Jose with no kids allowed. (max_steps=6)
run_id 4c80739baffd: steps=1 tool_calls=1 stop_reason=safety_block (3.84 s)
answer: I can't run that search. Blocked by fair-housing safety rule: listing searches cannot filter on a protected characteristic (familial status: 'no kids'). Search by location, title words, or available units instead.

[step ceiling] Compare landlords 1, 2, 3, 4 and 5. Call landlord_portfolio_stats separately for each of the five landlords, and only answer after you have all five results: which one manages the most listings? (max_steps=3)
run_id e96c59eb9802: steps=3 tool_calls=2 stop_reason=max_steps (18.59 s)
answer:

saved reports/hw05/raw/agent_runs.jsonl, agent_scenarios.json; updated METRICS.md
Kavan Siddesh data260-9486 %
```

Figure 35. run_agent with llama3.2:3b: steps, tool calls and stop reason per scenario. (p5_agent_runs.png)

III. Additional tests

Tests 14 (safety rule blocks a call) and 15 (run_agent with MockModel stops at max_steps) are shown passing in Figure 33.

IV. Metrics

Scenario	Input	Steps	Tool calls	Stop reason
search	Find up to 3 rental listings in Campbell.	2	1	completed
detail lookup	Give me the details for listing LST-00042.	2	1	completed
aggregate	How many listings does landlord 3 manage, and how many units are available in total?	2	1	completed
safety rule	Find apartments in San Jose with no kids allowed.	1	1	safety_block
step ceiling	Compare landlords 1, 2, 3, 4 and 5. Call landlord_portfolio_stats separately for each of the five landlords, and only answer after you have all five results: which one manages the most listings?	3	2	max_steps

V. Reflection

I picked run e96c59eb9802 ("step ceiling" scenario, max_steps = 3) from raw/agent_runs.jsonl. I asked the agent to compare landlords 1 to 5 by calling landlord_portfolio_stats once for each landlord and to answer only after it had all five results.

Step 1: llama3.2:3b replied with five tool calls in one message (landlord_id "1" to "5"). My harness runs one tool call per step, so it executed only the first call and logged the other four as ignored_tool_calls. execute_tool turned the string "1" into the integer 1 and returned ok: true: Farah Okafor, 269 listings, 270 available units.

Step 2: the model did not call a tool. Instead it wrote a JSON envelope in its text reply for "Samantha Lee" with 0 listings. That landlord does not exist, so the model invented a tool result. The text guard marked this as fabricated_tool_result, did not accept it as the final answer, and sent the model a correction asking it to call the real tool.

Step 3: the model called landlord_portfolio_stats with landlord_id "2" and got a real result: Hana Iyer, Rose Garden Realty, 279 listings.

The turn counter then reached 3, which equals max_steps, so the loop stopped with stop reason max_steps and no final answer (3 steps, 2 tool calls). The safety rule was not involved because no search query was sent, so safety_blocked is false on both tool calls.

This run showed me why the harness needs limits that do not depend on the model behaving well: the 3B model ignored "one at a time" and invented data. The step ceiling made sure the run still ended, and the log shows where it went wrong. For this kind of comparison I would raise max_steps or add a tool that compares landlords in one call.